SunScout

Light Tracker avec ESP32-WROOM

Document Technique de Rendu Final

Projet de Fin d'Année (PFA) — Département Sciences Informatiques

Faculté des Sciences de Tunis | Université Tunis El Manar

Projet	SunScout — Système de suivi solaire intelligent
Encadrant	M. Faouzi Moussa — FST, Département Sciences Informatiques
Classe	LCE2
Membres	Farah Ben Arbi • Baraa Ben Arfi • Rym Jlidi • Amani Arrouri • Mahdi Aouni
Date	19 Avril 2026
Fichier	Farahbenarbi_Baraabenarfi_Rymjlidi_Amaniarrouri_Mahdiaouni_RenduFinal.pdf
GitHub	rimjlidi228-web/SunScout_pfa

■ 1. Informations sur le Groupe et le Projet

1.1 Membres de l'équipe

	Nom & Prénom	Rôle dans le projet	Email
1	Mahdi Aouni	Développement embarqué ESP32 / Câblage	[mahdi.aouni@etudiant-fst.utm.tn]
2	Farah Ben Arbi	Dashboard Web (HTML/CSS/JS)	[farah.benarbi@etudiant-fst.utm.tn]
3	Baraa Ben Arfi	Programmation ESP32	[baraa.benarfi@etudiant-fst.utm.tn]
4	Rym Jlidi	documentation,GitHub	[rim.jlidi@etudiant-fst.utm.tn]
5	Amani Arrouri	Calibration, documentation	[ameni.arrouri@etudiant-fst.utm.tn]

1.2 Description du projet

SunScout est un système IoT de suivi solaire automatique basé sur la carte ESP32-WROOM-32. Il utilise deux capteurs LDR (Light Dependent Resistor) pour détecter la direction de la source lumineuse et commande un servomoteur SG90 afin d'orienter le panneau vers la lumière. Le système est alimenté par une batterie rechargeable avec un module de gestion d'alimentation. Un tableau de bord web interactif permet de visualiser les données en temps réel.

Élément	Détails
Projet	SunScout — Smart Solar Tracking System
Type	Projet IoT / Systèmes embarqués / Application Web embarquée
Matériel	ESP32-WROOM-32, 2× LDR(capteurs de luminosité), Servo SG90, LED, Résistances 220Ω, panneau solaire (5V – 1W/2W) ,Batterie 18650(3.7V) rechargeable, Module de charge TP4056
Logiciel	Arduino IDE, HTML/CSS/JavaScript, API REST (JSON), Wokwi (simulation), Git / GitHub (gestion de version), Jira (gestion de projet)
Communication	WiFi 802.11 b/g/n — API HTTP GET /data
Cadre	Projet de Fin d'Année(PFA) — Département Sciences Informatiques, FST, 2025-2026

■ 2. Lien du projet

2.1 Dépôt GitHub — Code source

Dépôt GitHub [rimjlidi228-web/SunScout_pfa]

Le dépôt contient :

- sunscout_esp32/sunscout_esp32.ino — Code Arduino complet pour l'ESP32
- dashboard/index.html — Interface web du tableau de bord
- README.md — Documentation d'installation et d'utilisation

3. Consignes d'Exécution

3.1 Prérequis matériels

Composant	Référence	Qté
Carte microcontrôleur	ESP32-WROOM-32	1
Capteur luminosité	LDR GL5528 + résistance 10 kΩ	2
Servomoteur	SG90 (180°, 4.8–6V)	1
LED indicatrice	LED 5mm (rouge)	1
Câble USB	USB-A vers Micro-USB DATA (pas power-only)	1
Alimentation	5V via USB PC ou adaptateur	1
Module de charge	TP4056	1

Batterie	18650 (3.7V)	1
Panneau solaire	5V – 1W/2W	1
Structure mécanique	Support panneau avec axe servo	1
Résistances	220Ω (pour LED), 10kΩ (pour diviseur LDR)	3

3.2 Prérequis logiciels

-

Arduino IDE	2.x (recommandé)	https://www.arduino.cc/en/software
Driver CP2102 (Silicon Labs)	VCP 6.7+	https://www.silabs.com/developers/usb-to-uart-bridge-vcp-driver
Driver CH340 (si clone)	CH341SER	http://www.wch-ic.com/downloads/CH341SER_EXE.html
Arduino IDE	2.x (recommandé)	https://www.arduino.cc/en/software
Package esp32 (Espressif)	2.x	Via Gestionnaire de cartes Arduino IDE
Bibliothèque ESP32Servo	1.x	Via Gestionnaire de bibliothèques Arduino IDE
Bibliothèque WebServer	incluse ESP32	Incluse dans le package ESP32 Espressif
Bibliothèque WiFi (ESP32)	incluse ESP32	Incluse dans le package ESP32 Espressif
Visual Studio Code	dernière	https://code.visualstudio.com/
Navigateur web	Chrome/Firefox/EdgeInclus dans l'OS	Inclus dans le système

3.3 Étapes d'installation et d'exécution

Étape 1 — Installer le driver USB

Brancher l'ESP32 à l'ordinateur, puis ouvrir le **Gestionnaire de périphériques Windows**.
Si un périphérique inconnu apparaît, installer le driver approprié selon le composant USB utilisé :

- CP2102 (Silicon Labs)
- CH340 (pour les cartes clones ESP32)

Après installation, un port **COMx** doit apparaître dans la section *Ports (COM & LPT)*.

Étape 2 — Configurer Arduino IDE

Fichier → Préférences → coller dans 'URL gestionnaire de cartes supplémentaires' : https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json. Puis Outils → Gestionnaire de cartes → chercher 'esp32' → Installer.

Étape 3 — Sélectionner la carte et le port

Dans le menu **Outils** :

- Type de carte : **ESP32 Arduino → ESP32 Dev Module**
- Port : sélectionner le port **COMx** détecté

Étape 4 — Installer la bibliothèque ESP32Servo

Outils → Gérer les bibliothèques → chercher 'ESP32Servo' → Installer la version de Kevin Harrington.

Étape 5 — Configurer le réseau WiFi dans le code

Ouvrir sunscout_esp32.ino. Modifier :

const char* ssid = "VOTRE_WIFI";

const char* password = "VOTRE_MOT_DE_PASSE";

Étape 6 — Téléverser le code

Cliquer sur la flèche 'Téléverser' (Ctrl+U). Si erreur 'espcomm_sync failed' : maintenir le bouton BOOT de l'ESP32 pendant le téléversement.

Étape 7 — Récupérer l'adresse IP

Ouvrir le Moniteur Série (Ctrl+Maj+M) à 115200 bauds. L'adresse IP de l'ESP32 s'affiche, ex : 'IP: 192.168.1.45'. La noter.

Étape 8 — Lancer le dashboard

Ouvrir dashboard/index.html dans un navigateur. Modifier la variable ESP32_IP dans app.js avec l'IP notée à l'étape 7. Le dashboard affiche les données en temps réel.

■ Utiliser impérativement un câble USB avec fils de données (pas un câble de charge seul). Un câble power-only ne permet pas le téléversement du code.

■ 4. Documentation Technique

4.1 Architecture du système

Couche	Composants	Rôle
Couche Capteurs (IoT)	2× LDR + résistances 10kΩ GPIO 34 (gauche), GPIO 35 (droite)	Mesure de l'intensité lumineuse sur deux côtés
Couche Logique (ESP32)	ESP32-WROOM-32 Arduino (IDE / C++)	Lecture ADC, calcul différentiel, commande servo, serveur HTTP
Couche Actionneurs	Servo SG90 (GPIO 13) LED (GPIO 4) Module TP4056 + Batterie 18650	Orientation physique du panneau + indicateur visuel + stockage énergie
Couche Présentation	HTML / CSS / JavaScript +Fetch API (HTTP GET)	Dashboard web — visualisation des données en temps réel

4.2 Brochage ESP32

GPIO	Type	Connecté à	Description
GPIO 34	Entrée analogique (ADC1_CH6)	LDR Gauche	Lecture 0–4095 (12 bits)
GPIO 35	Entrée analogique (ADC1_CH7)	LDR Droite	Lecture 0–4095 (12 bits)
GPIO 13	Sortie PWM	Servo SG90 (signal)	Contrôle de l'angle (0°–180°)
GPIO 4	Sortie numérique	LED + résistance	Indicateur d'état (HIGH = actif)
3.3V	Alimentation	LDR + résistances	Référence du diviseur de tension
GND	Masse commune	Tous composants	Masse commune du système
VIN (5V)	Alimentation	Servo SG90 (Vcc)	Alimentation servo

4.3 Algorithme de suivi solaire

sunscout_esp32.ino — Logique principale (boucle)

```
void loop() {
server.handleClient();

// LDR READ
left = 4095  - analogRead(LDR_LEFT);
right = 4095 - analogRead(LDR_RIGHT);
Serial.print("LDR Left: "); Serial.print(left);
Serial.print(" | LDR Right: "); Serial.print(right);
Serial.print(" | Position: "); Serial.println(position);

// SERVO CONTROL
int diff = left - right;
if (abs(diff) > threshold) {
if (diff > 0) { position -= speedstep; }
else { position += speedstep; }
position = constrain(position, 0, 180);
myServo.write(position);
}
// LED CONTROL — allumée si lumière forte sur au moins un capteur
if (left < 2000 || right < 2000) {
digitalWrite(LED_PIN, HIGH); ledState = true;
} else {
digitalWrite(LED_PIN, LOW); ledState = false;
}
delay(80);
}
```

4.4 API REST embarquée

■ **Endpoint : GET http://[ESP32_IP]/data**

```
// Exemple de réponse JSON :
{
"left": 2800, // Valeur LDR gauche (0-4095)
"right": 2100, // Valeur LDR droite (0-4095)
"angle": 95, // Position servo en degrés (0-180)
"led": true // LED allumée = lumière détectée
}
```

4.5 Paramètres configurables

Constante	Valeur par défaut	Description
LDR_LEFT_PIN	34	GPIO du capteur LDR gauche
LDR_RIGHT_PIN	35	GPIO du capteur LDR droite
SERVO_PIN	13	GPIO de commande PWM du servo
LED_PIN	4	GPIO de la LED indicatrice
SERVO_MIN	0°	Angle minimal du servo (butée gauche)
SERVO_MAX	180°	Angle maximal du servo (butée droite)
threshold	200	Seuil différentiel LDR pour déclencher le mouvement
speedstep	2°	Pas de rotation à chaque cycle
loop_delay	100 ms	Fréquence de mise à jour (≈10 Hz)

■ 4.6 Environnement Logiciel

Le développement du système SunScout repose sur un ensemble d'outils logiciels permettant d'assurer le bon fonctionnement du système de suivi solaire ainsi que la visualisation en temps réel des données.

- **Environnement de développement embarqué — Arduino IDE**

  Le programme embarqué a été développé à l'aide de **Arduino IDE 2.x**, qui permet d'écrire, compiler et téléverser le code sur la carte ESP32. Cet environnement est utilisé pour programmer le microcontrôleur ESP32, gérer les capteurs et actionneurs, et configurer la communication WiFi.

- **Développement Web (Frontend) — Visual Studio Code**

  L'interface utilisateur a été réalisée avec **HTML, CSS et JavaScript** via Visual Studio Code. Elle permet d'afficher les données des capteurs LDR, de visualiser la position du panneau solaire, de suivre l'état du système en temps réel et d'afficher des graphiques dynamiques. Un dashboard moderne et responsive a été conçu pour assurer une supervision claire et intuitive.

- **Simulateur — Wokwi**

  Wokwi a été utilisé pour simuler le circuit du système SunScout avant sa réalisation physique. Cette simulation a permis de valider le câblage, la logique du programme et le fonctionnement des capteurs et du servomoteur.

Bibliothèques Arduino utilisées

Bibliothèque	Rôle	Installation
WebServer	Permet de créer un serveur HTTP sur l'ESP32, gérer les requêtes HTTP, exposer l'API `/data` et envoy au client	Incluse dans le package ESP32 (Espressif)
WiFi (ESP32 Core)	Permet à l'ESP32 de se connecter à un réseau WiFi et d'agir comme client ou point d'accès	Incluse dans le package ESP32 (Espressif)
ESP32Servo	Permet de contrôler le servomoteur SG90 via un signal PWM et d'orienter le panneau solaire	Gestionnaire de bibliothèques Arduino IDE (Kevin Harrington)
ArduinoJson (optionnel)	Facilite la génération et la sérialisation des données en format JSON pour l'API REST	Gestionnaire de bibliothèques Arduino IDE

■ 4.7 Environnement Matériel

Le système SunScout repose sur plusieurs composants matériels permettant d'assurer le suivi solaire ainsi que la production et le stockage de l'énergie.

- **Microcontrôleur ESP32-WROOM-32**
  Cœur du système. Traite les données des capteurs LDR, contrôle le servomoteur SG90 via PWM et assure la communication WiFi. Embarque un serveur HTTP qui expose l'API REST /data. Alimentation : 5V via USB. Fréquence CPU : 240 MHz dual-core.

- **Capteurs LDR GL5528**
  Deux capteurs photorésistifs mesurent l'intensité lumineuse gauche et droite. Montés en diviseur de tension avec résistances 10kΩ. Lecture ADC 12 bits (0–4095). Plus l'intensité lumineuse est élevée, plus la résistance diminue.Une correction logicielle est appliquée : valeur_corrigée = 4095 - analogRead(pin).

- **Servomoteur SG90**
  Assure la rotation du panneau solaire entre 0° et 180°. Commandé par signal PWM depuis GPIO 13 de l'ESP32 via la bibliothèque ESP32Servo. Alimentation : 5V (VIN). Couple : 1.8 kg/cm. Résolution : 2° (speedstep configurable).

• **Panneau solaire 5V – 1W/2W** Capte l'énergie solaire et la convertit en énergie électrique. Alimenté par le rayonnement direct ou indirect. Connecté au module TP4056 pour la charge de la batterie.
• **Module de charge TP4056 + Batterie 18650** Assure la gestion d'alimentation du système. Le TP4056 recharge la batterie 18650 (3.7V) depuis le panneau solaire. La batterie alimente l'ESP32 et les composants en l'absence de connexion USB.
• **LED indicatrice + résistance 220Ω** LED 5mm (rouge ou verte) connectée sur GPIO 4 avec résistance de limitation 220Ω. S'allume quand la lumière est détectée (valeur LDR < 2000 sur au moins un capteur). Indique l'état actif du système de suivi.
• **Structure mécanique** Supporte le panneau solaire et le servomoteur. Assure l'axe de rotation horizontal pour le suivi mono-axial. Les deux capteurs LDR sont montés de part et d'autre d'un séparateur opaque pour maximiser la différentialité.

■ 4.8 Architecture de l'Application (3-Tiers)

Le système SunScout repose sur une architecture **3-Tiers**, assurant une séparation claire des responsabilités :

Tier	Composants	Responsabilités
Tier Présentation (Frontend)	HTML / CSS / JavaScript Navigateur web	• Visualisation données temps réel • Affichage capteurs LDR • Suivi angle panneau solaire • Graphique historique • Journal des mouvements • Indicateur Jour/Nuit
Tier Applicatif (Backend embarqué)	ESP32-WROOM-32 Arduino C++ WebServer, WiFi, ESP32Servo	• Lecture capteurs ADC • Calcul différentiel LDR • Contrôle PWM servomoteur • Serveur HTTP embarqué • Exposition API REST /data
Tier Données (IoT Sensors)	LDR Module TP4056 Batterie 18650	• Génération données lumineuses • Stockage énergie solaire • Transmission JSON temps réel • Format : {left, right, angle, led}

■ 5. Guide Utilisateur

Ce guide décrit comment utiliser le système SunScout une fois installé et le code téléversé sur l'ESP32.

5.1 Démarrage du système

1. Brancher l'ESP32 sur une alimentation USB (PC ou adaptateur 5V).
2. La LED s'allume brièvement : l'ESP32 démarre et se connecte au WiFi.

3. Après quelques secondes, l'ESP32 est prêt. La LED clignote si de la lumière est détectée.

4. Ouvrir dashboard/index.html dans un navigateur sur le même réseau WiFi.

5. Le dashboard affiche automatiquement les données en temps réel (rafraîchissement 1s).

5.2 Interface du dashboard — Éléments affichés

Élément du dashboard	Description	Comportement
LDR Gauche	Barre de progression (0–100%)	Augmente avec la lumière gauche
LDR Droite	Barre de progression (0–100%)	Augmente avec la lumière droite
Angle Servo	Valeur numérique en degrés (0–180°)	Mise à jour dynamique
Indicateur Jour/Nuit	Icône soleil ou lune	Basé sur la moyenne des deux LDR
Efficacité suivi	Pourcentage d'alignement	100% = panneau parfaitement centré
Graphique temps réel	Courbe LDR gauche + droite + angle	Historique des 60 dernières secondes
Journal mouvements	Liste horodatée des changements	Enregistre chaque mouvement du servo
LED virtuelle	Indicateur ACTIVE / VEILLE	Reflète l'état réel de la LED hardware
RSSI WiFi	Force du signal WiFi (dBm)	Aide au diagnostic de connexion

5.3 Comportement attendu du système

Fonctionnement normal

- La lumière placée à gauche → le servo tourne vers la gauche
- La lumière placée à droite → le servo tourne vers la droite
- La lumière placée au centre → le servo reste immobile (différence < seuil)
- Lumière éteinte ou très faible → LED s'éteint, système en veille
- Le dashboard affiche les valeurs mises à jour chaque seconde

5.4 Conseils d'utilisation optimale

- Séparer les deux LDR par un petit séparateur opaque (carton noir) pour maximiser la différentialité
- Placer le système en extérieur ou près d'une fenêtre pour un suivi solaire réel
- Éviter les sources lumineuses parasites (lampes néon, reflets) qui peuvent perturber les capteurs
- Utiliser un câble USB DATA pour le téléversement — les câbles power-only ne fonctionnent pas • Vérifier le RSSI WiFi dans le dashboard pour garantir une connexion stable

■ 6. Difficultés Rencontrées & Solutions

■ Endommagement du module électronique

Problème : Détérioration d'un composant électronique due à une mauvaise gestion de l'alimentation.

Solution : Réduction et stabilisation de l'alimentation fournie au module afin de respecter ses spécifications électriques. Ajout de protections matérielles (résistances, régulateur de tension).

■ Connexion WiFi instable

Problème : L'ESP32 perdait parfois la connexion WiFi, rendant le dashboard inaccessible.

Solution : Ajout d'une vérification WiFi.status() dans la boucle principale. Reconnexion automatique si perte de signal. Affichage du RSSI dans le dashboard.

■ Inversion des valeurs LDR

Problème : Plus il y a de lumière, plus la résistance LDR diminue, ce qui donnait des valeurs contre-intuitives.

Solution : Inversion logicielle : valeur_corrigée = 4095 - analogRead(pin), ce qui donne une valeur haute pour une forte lumière.

■ Calibration des capteurs LDR

Problème : Les deux capteurs LDR n'avaient pas exactement la même sensibilité, causant un biais de direction.

Solution : Ajout d'un seuil de tolérance (threshold = 200) pour éviter les oscillations dues au bruit. Normalisation des valeurs pour une échelle cohérente (0–100%).

■ Stabilité du servomoteur

Problème : Des oscillations du servomoteur étaient observées à cause de petites fluctuations des capteurs.

Solution : Augmentation du seuil de tolérance et réduction du pas de rotation (speedstep = 2°) pour un mouvement plus fluide et stable.

■ Synchronisation hardware / dashboard

Problème : Des décalages entre les données réelles et l'affichage web étaient constatés.

Solution : Optimisation du polling HTTP (toutes les 1 seconde) et gestion des timeouts fetch() pour garantir des données fraîches sans surcharger l'ESP32.

■ 7. Améliorations Possibles

- Suivi solaire biaxial
- Communication temps réel (WebSockets)
- Normalisation (0–100%)
- Filtrage numérique (moyenne glissante)
- Capteur énergie (INA219)
- Sécurité API (authentification)
- Cloud (MQTT / IoT)
- Deep Sleep (économie énergie)

8.1 Tous les liens du projet

Code source (GitHub)
rimjlidi228-web/SunScout_pfa

8.2 Structure du dépôt GitHub

```
SunScout/
■■■ sunscout_esp32/
■ ■■■ sunscout_esp32.ino ← Code Arduino principal (ESP32)
■■■ dashboard/
■ ■■■ index.html ← Interface web du tableau de bord
■■■ README.md ← Documentation installation et utilisation
```

8.3 Ressources et bibliographie

Source	Référence / Lien
Arduino Documentation	https://docs.arduino.cc/
Espressif ESP32 Docs	https://docs.espressif.com/
Wokwi Simulator	https://wokwi.com/
Random Nerd Tutorials	https://randomnerdtutorials.com/
Last Minute Engineers	https://lastminuteengineers.com/
BANZI & SHILOH, Getting Started with Arduino	O'Reilly Media, 2015
MONK, Programming Arduino: Getting Started	McGraw-Hill, 2016
YADAV et al., Solar tracking system: A review	Renewable & Sustainable Energy Reviews, 2013